Software Discovery & Entitlement Reconciliation (SCCM / Intune / Flexera)

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Software Discovery & Entitlement Reconciliation (SCCM / Intune / Flexera) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Software Discovery & Entitlement Reconciliation 5

What this solution is . . . 5

The named artifacts . . . 5

How the pipeline works . . . 5

Why discovery sources differ (and why normalization matters) . . . 6

What you can control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Software Discovery & Entitlement Reconciliation 7

2.1 At least one installed-software discovery source . . . 7

2.2 Read access to each discovery source . . . 7

2.3 An entitlement system of record . . . 8

2.4 A software normalization approach . . . 8

2.5 A defined reconciliation scope and pilot source . . . 8

2.6 Reporting toolchain . . . 8

2.7 (Optional) Microsoft Graph PowerShell for Intune discovery . . . 8

2.8 (Optional) Configuration Manager access for the scripted path . . . 8

Checklist . . . 9

3. Implementation Guide — Software Discovery & Entitlement Reconciliation 9

Step 1 — Define the pilot scope . . . 9

Step 2 — Confirm inventory is enabled on the pilot source . . . 9

Step 3 — Export the installed-software data . . . 10

Step 4 — Build or update the normalization map . . . 10

Step 5 — Import the entitlement register . . . 10

Step 6 — Reconcile and produce the position . . . 10

Step 7 — Route exceptions . . . 11

Step 8 — Remediate mapping gaps safely . . . 11

Step 9 — Validate the pilot . . . 11

Step 10 — Expand to all sources . . . 11
5. Testing Guide — Software Discovery & Entitlement Reconciliation 12
Test 1 — Discovery returns installed software . . . 12
Test 2 — Every raw row normalizes to a SKU . . . 12
Test 3 — Entitlement register joins on the SKU vocabulary . . . 12
Test 4 — Reconciliation ledger is produced . . . 12
Test 5 — Exceptions are captured, not dropped . . . 12
Test 6 — Remediation preview is safe . . . 12
Test 7 — Position holds across sources . . . 12
Pass/fail gate . . . 13
6. Troubleshooting Guide — Software Discovery & Entitlement Reconciliation 13
A. Discovery returns no installed software . . . 13
B. Rows do not normalize to a SKU . . . 13
C. Reconciliation ledger looks wrong . . . 14
D. Exceptions never shrink . . . 14
E. Remediation made no change . . . 14
F. When to escalate . . . 14
7. Best Practices — Software Discovery & Entitlement Reconciliation 14
8. Deliverables Summary — Software Discovery & Entitlement Reconciliation 15
Documentation . . . 15
PowerShell scripts . . . 16
The named artifacts . . . 16
What the buyer receives . . . 16
Outcome . . . 17
Appendix A — PowerShell Scripts 18

Executive Summary — Software Discovery & Entitlement Reconciliation (SCCM / Intune / Flexera)

Discovery tools tell you what is installed. They do not tell you whether you own it. Raw inventory arrives as thousands of unnormalized titles, editions, and versions that map to no SKU and link to nothing you actually bought — so the question that matters, "are we compliant and where can we reharvest?", goes unanswered.

This package turns software discovery into a reconciled license position. It is a complete, do-it-today method for pulling installed-software data from Configuration Manager, Microsoft Intune, or a discovery tool such as Flexera, normalizing the noise into clean SKUs, mapping those SKUs to your entitlements, and producing the reconciliation that drives compliance decisions and reharvesting. It is written for a Software Asset Management (SAM) analyst with no prior reconciliation pipeline.

What you get

- A plain-language Solution Overview of discovery sources, software normalization, the entitlement reconciliation model, and how the output feeds compliance and optimization.
- A Requirements Guide with verify / implement-if-missing for every prerequisite. Vendor-specific Flexera setup is deferred to the vendor's documentation.
- An Implementation Guide of precise, ordered steps: confirm inventory, export installed-software data, normalize titles/editions/versions to SKUs, map to entitlements, reconcile and flag exceptions, then pilot, validate, and expand to more sources.
- Five PowerShell scripts for installed-software discovery, normalization/mapping validation, a reconciliation and exception report, safe remediation of mapping gaps, and a discovery-scope group builder.
- Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A repeatable reconciliation pipeline: discovery from one or more sources, normalized into a single SKU vocabulary, matched against an authoritative entitlement count, with every unmatched install routed to an exception register for decision. Proven on a one-source pilot before it scales.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Raw software discovery turned into a clean, reconciled license position — normalized, mapped to entitlements, exceptions flagged — the foundation for compliance and reharvesting, proven on a pilot before you scale.

1. Solution Overview — Software Discovery & Entitlement Reconciliation

What this solution is

Software discovery and entitlement reconciliation is the core of Software Asset Management (SAM): it takes the raw list of what is installed across your estate, cleans that list into a controlled SKU vocabulary, and matches it against what you have actually purchased. The output is your effective license position — where you are over-deployed (compliance risk), where you are under-deployed (reharvesting and savings opportunity), and which installs cannot be explained (exceptions).

This is a repeatable method that is source-agnostic: it works whether your installed-software data comes from Microsoft Configuration Manager (SCCM), Microsoft Intune detected apps, or a dedicated discovery tool such as Flexera. The pipeline normalizes all sources into the same SKU vocabulary, so the reconciliation logic stays identical regardless of where the inventory originated.

The named artifacts

This solution produces and reuses a fixed set of named artifacts. Every step references them by name, and the scripts read and write them. Each name follows the pattern `<Purpose> – <Scope> – <Stage>`:

- `Installed-Software Inventory – All Sources – Discovery` — the merged raw extract of installed

software from every in-scope discovery source (Configuration Manager, Intune, Flexera export).

- `Normalization Map – SKU Vocabulary – Configure` — the lookup that translates raw publisher,

title, edition, and version strings into a single canonical SKU.

- `Entitlement Register – Systems of Record – Configure` — the authoritative count of purchased

entitlements per SKU, sourced from the entitlement system of record (Global Requirement G3).

- `Reconciliation Ledger – SKU vs Entitlement – Validate` — the matched output: installed quantity

per SKU versus entitled quantity per SKU, with the resulting position.

- `Exception Register – Unmatched & Edge Cases – Validate` — every install that did not map to a SKU,

or mapped to a SKU with no entitlement, routed for a decision.

- `Discovery Scope Group – Pilot Source – Pilot` — the targeting group (one source / one product

family) that bounds the pilot before expansion.

How the pipeline works

1. Discovery sources are queried for installed software and merged into the

`Installed-Software Inventory – All Sources – Discovery`. Configuration Manager supplies its software inventory and Asset Intelligence data; Intune supplies its detected apps list; Flexera supplies a normalized installed-software export from its own console.

1. The raw rows are run through the `Normalization Map – SKU Vocabulary – Configure`, which collapses

the many ways one product appears (different publisher spellings, edition suffixes, version and patch levels) into one canonical SKU.

1. Normalized installs are aggregated per SKU and matched against the

`Entitlement Register – Systems of Record – Configure` to build the `Reconciliation Ledger – SKU vs Entitlement – Validate`.

1. Anything that cannot be matched — an install whose title resolves to no SKU, or a SKU with installs

but no entitlement — is written to the `Exception Register – Unmatched & Edge Cases – Validate` for a SAM analyst decision (add a normalization rule, record a missing entitlement, or flag a true compliance gap).

1. The ledger and exception register drive action: true-up where over-deployed, reharvest where

under-deployed, and resolve exceptions so the next run is cleaner.

Why discovery sources differ (and why normalization matters)

Each source describes the same installed product differently. Configuration Manager reports from installed-product inventory and Asset Intelligence; Intune reports a detected-apps list keyed on display name, publisher, and version; Flexera applies its own Technopedia-based recognition before export. Left raw, the same product appears as many distinct rows and cannot be counted. Normalization to a single SKU vocabulary is what makes counting — and therefore reconciliation — possible.

What you can control

- Which discovery sources are in scope and how they are merged.
- The normalization rules that map titles, editions, and versions to SKUs.
- The entitlement counts the position is measured against.
- How exceptions are triaged and who owns each decision.
- The pilot scope (one source or one product family) before fleet-wide reconciliation.

Benefits

- Compliance — a defensible, reproducible license position for audits and true-ups.
- Savings — under-deployment surfaced for reharvesting and renewal right-sizing.
- Trust — a single SKU vocabulary so every source counts consistently.
- Auditability — every unmatched install is captured and decided, not silently dropped.

• Repeatability — the same pipeline runs each cycle and on any source.

Limitations (be honest with buyers)

• This solution reconciles installed-software discovery to entitlements; it does not measure SaaS user-based consumption (that is a separate discipline).

• Normalization quality depends on the rules you maintain; a new product family needs a new rule.

• Flexera-specific configuration (connectors, inventory beacons, Technopedia, the export itself) is a vendor product and is deferred to Flexera's own documentation — this solution consumes its export.

• Metric-based licensing (per-core, per-processor, per-user with downgrade rights) requires the entitlement register to record the metric; the pipeline reconciles counts against the metric you record, it does not infer entitlement rules.

The standard deployment model

> *Confirm a discovery source and an entitlement register → build the normalization map and import entitlements → pilot on one source / one product family → validate the reconciliation and exceptions → expand to all sources. Pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***

2. Requirements Guide — Software Discovery & Entitlement Reconciliation

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-asset-mgt.md`.

2.1 At least one installed-software discovery source

Description: A source that reports installed software per device — Configuration Manager (SCCM), Microsoft Intune detected apps, or a discovery tool such as Flexera. **Why:** The reconciliation pipeline cannot start without an installed-software extract to normalize. **Verify:** Run `SWDISC-Discovery.ps1` against one source and confirm it returns installed-software rows; or in Configuration Manager open `Assets and Compliance > Asset Intelligence`, or in Intune open `Apps > Monitor > Discovered apps`. **Implement:** Enable Configuration Manager software inventory / Asset Intelligence, or confirm Intune detected apps are populated, or onboard a discovery tool. Flexera-specific setup is deferred to Flexera's documentation.

2.2 Read access to each discovery source

Description: Rights to query or export installed-software data from each in-scope source. **Why:** Discovery is read-only, but it still requires query/export permission. **Verify:** For Intune, you can run a Microsoft Graph query with `DeviceManagementManagedDevices.Read.All`; for Configuration Manager, the SMS Provider / WMI namespace is reachable with a read role; for Flexera, you can produce an installed-software export from its console. **Implement:** Assign a least-privilege read role per source (e.g. an Intune read role, a Configuration Manager Read-only Analyst, or the appropriate Flexera role). Reserve

elevated roles for changes.

2.3 An entitlement system of record

Description: An authoritative record of purchased entitlements per product, used to build the `Entitlement Register – Systems of Record – Configure`. **Why:** A position can only be computed against a known, owned entitlement count. **Verify:** You can locate the current entitlement quantity (and licensing metric) for a representative product. Covered category-wide by Global Requirement G3. **Implement:** Stand up the entitlement repository and import current purchase orders / true-up statements; record the licensing metric per SKU.

2.4 A software normalization approach

Description: A maintained `Normalization Map – SKU Vocabulary – Configure` that translates raw publisher/title/edition/version strings into canonical SKUs. **Why:** Raw inventory lists the same product many ways; without normalization, installs cannot be counted or matched. **Verify:** Run `SWDISC-Validation.ps1` and confirm a normalization map is present and that a sample of raw titles resolves to SKUs. **Implement:** Create the normalization map (a controlled lookup table) seeded from your top products; add a rule for each new product family as it appears in discovery.

2.5 A defined reconciliation scope and pilot source

Description: A bounded `Discovery Scope Group – Pilot Source – Pilot` — one source or one product family — to prove the pipeline before fleet-wide reconciliation. **Why:** Piloting on one source validates accuracy and surfaces normalization gaps cheaply. Aligned with Global Requirement G7. **Verify:** A pilot source/product family and its data owner are identified; build it with `SWDISC-Set-ScopeTargeting.ps1`. **Implement:** Choose a high-value, well-understood source or product family as the first pass.

2.6 Reporting toolchain

Description: Excel (Power Query) and/or Power BI to view the `Reconciliation Ledger – SKU vs Entitlement – Validate` and the `Exception Register – Unmatched & Edge Cases – Validate`. **Why:** The position and exceptions are delivered as reports for IT, procurement, and finance. **Verify:** Excel Power Query opens a CSV export; Power BI Desktop can connect. Covered category-wide by Global Requirement G5. **Implement:** Install a current Excel and/or Power BI Desktop.

2.7 (Optional) Microsoft Graph PowerShell for Intune discovery

Description: The Microsoft Graph PowerShell SDK, used by the scripts to read Intune detected apps. **Why:** Required only when Intune is an in-scope discovery source for the scripted path. **Verify:** `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns the module. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser` (the scripts install it if missing).

2.8 (Optional) Configuration Manager access for the scripted path

Description: The Configuration Manager console / cmdlets or WMI access to the site's `root\sms\site_<code>` namespace, used by the scripts to read software inventory. **Why:** Required only when Configuration Manager is an in-scope discovery source for the scripted path. **Verify:** `Get-CMSite` succeeds in the console, or the SMS Provider WMI namespace is reachable. **Implement:** Install the Configuration Manager console (which provides the cmdlets) or grant read access to the SMS Provider WMI namespace.

Checklist

- At least one discovery source returns installed software — `SWDISC-Discovery.ps1`
- Read access confirmed per in-scope source
- Entitlement system of record located (G3)
- Normalization map present and resolving samples — `SWDISC-Validation.ps1`
- Pilot source / product family defined — `SWDISC-Set-ScopeTargeting.ps1`
- Reporting toolchain available (G5)
- (Optional) Microsoft Graph PowerShell for Intune discovery
- (Optional) Configuration Manager console/WMI access for the scripted path

3. Implementation Guide — Software Discovery & Entitlement Reconciliation

Follow these steps **in order**. Steps run partly in your discovery consoles (Configuration Manager, the Microsoft Intune admin center at `https://intune.microsoft.com`, or the Flexera console) and partly with the supplied scripts. Do this — the reasoning lives in the Solution Overview.

> ***Model:*** *Pilot on one source / one product family, validate the reconciliation, then expand to all sources. The artifacts you build are reused verbatim every cycle.*

Step 1 — Define the pilot scope

1. Choose one discovery source (or one product family within it) as the

`Discovery Scope Group – Pilot Source – Pilot`.

1. Build the scope targeting with `SWDISC-Set-ScopeTargeting.ps1` (creates/confirms the pilot scope

group for the chosen source/product family). Record the data owner.

Step 2 — Confirm inventory is enabled on the pilot source

1. Configuration Manager: open `Administration > Client Settings`, confirm `Software Inventory` and

`Hardware Inventory` (with Asset Intelligence reporting classes) are enabled for the scoped collection.

1. Intune: open `Apps > Monitor > Discovered apps` and confirm the detected-apps list is populated for

the scoped devices.

1. Flexera: confirm inventory is flowing in the Flexera console. Flexera-specific configuration

(beacons, connectors, Technopedia recognition) is deferred to Flexera's documentation.

Step 3 — Export the installed-software data

1. Run `SWDISC-Discovery.ps1` against the pilot source to produce the

`Installed-Software Inventory – All Sources – Discovery` (read-only).

• For Intune, the script reads detected apps via Microsoft Graph.

• For Configuration Manager, the script reads software inventory via the Get-CM cmdlets / the SMS

Provider WMI namespace.

• For Flexera, place the installed-software export produced from the Flexera console where the

script expects it (the script consumes the export; it does not call a Flexera API).

1. Confirm the inventory file contains publisher, title, edition/version, and device columns.

Step 4 — Build or update the normalization map

1. Open the `Normalization Map – SKU Vocabulary – Configure` (a controlled lookup table).
2. For each distinct raw publisher/title/edition/version in the inventory, add or confirm a rule that

resolves it to one canonical SKU (collapse edition suffixes and version/patch levels into the SKU you license against).

1. Run `SWDISC-Validation.ps1` to confirm every raw row in the pilot inventory resolves to a SKU; rows

that do not resolve are reported so you can add rules.

Step 5 — Import the entitlement register

1. From the entitlement system of record (Global Requirement G3), build the

`Entitlement Register – Systems of Record – Configure`: one row per SKU with the entitled quantity and the licensing metric.

1. Confirm the register's SKUs use the **same** canonical SKU vocabulary as the normalization map (so

they join cleanly).

Step 6 — Reconcile and produce the position

1. Run `SWDISC-Report.ps1` to aggregate normalized installs per SKU and match them against the

`Entitlement Register – Systems of Record – Configure`, producing the `Reconciliation Ledger – SKU vs Entitlement – Validate` (read-only).

1. The ledger shows, per SKU: installed quantity, entitled quantity, and the position

(over-deployed / under-deployed / balanced).

Step 7 — Route exceptions

1. `SWDISC-Report.ps1` also writes the `Exception Register – Unmatched & Edge Cases – Validate`:

installs that resolved to no SKU, and SKUs with installs but no entitlement.

1. For each exception, take the SAM analyst decision: add a normalization rule (Step 4), record a

missing entitlement (Step 5), or flag a true compliance gap for procurement.

1. Re-run `SWDISC-Report.ps1` after resolving exceptions so the ledger reflects the decisions.

Step 8 — Remediate mapping gaps safely

1. Run `SWDISC-Remediation.ps1` to apply queued normalization-map fixes (e.g. add pending alias rules

for unmatched titles). It defaults to `-WhatIf` and only previews changes.

1. Review the preview, then re-run with `-WhatIf:$false` to commit the map updates. Re-run

`SWDISC-Validation.ps1` to confirm the previously unmatched rows now resolve.

Step 9 — Validate the pilot

1. `SWDISC-Validation.ps1` confirms all pilot rows normalize to SKUs.
2. `SWDISC-Report.ps1` confirms a clean ledger and a triaged exception register for the pilot SKUs.
3. Complete the Testing Guide and confirm the pilot pass gate before expanding.

Step 10 — Expand to all sources

1. Add the remaining discovery sources to the

`Installed-Software Inventory – All Sources – Discovery` (re-run `SWDISC-Discovery.ps1` per source and merge).

1. Extend the normalization map for products that only appear in the added sources (Step 4).
2. Re-run `SWDISC-Report.ps1` for the full scope; reconcile and triage exceptions each cycle.

Order of operations

`Pilot scope → confirm inventory → export discovery → normalization map → import entitlements → reconcile → route exceptions → remediate map gaps → validate pilot → expand to all sources`

5. Testing Guide — Software Discovery & Entitlement Reconciliation

Run after the pilot pipeline completes (Implementation Step 9) and after each reconciliation cycle or when a new source is added.

Test 1 — Discovery returns installed software

Do: Run `SWDISC-Discovery.ps1` against the pilot source. **Expected:** The `Installed-Software Inventory – All Sources – Discovery` is produced with publisher, title, edition/version, and device columns populated for the scoped devices.

Test 2 — Every raw row normalizes to a SKU

Do: Run `SWDISC-Validation.ps1` over the pilot inventory. **Expected:** All raw rows resolve through the `Normalization Map – SKU Vocabulary – Configure` to a canonical SKU; zero unresolved rows (or the unresolved rows are listed for rule creation).

Test 3 — Entitlement register joins on the SKU vocabulary

Do: Run `SWDISC-Validation.ps1` and confirm SKUs in the `Entitlement Register – Systems of Record – Configure` match the normalization map's vocabulary. **Expected:** Register SKUs and normalized SKUs share one vocabulary; no orphaned-vocabulary warnings.

Test 4 — Reconciliation ledger is produced

Do: Run `SWDISC-Report.ps1`. **Expected:** The `Reconciliation Ledger – SKU vs Entitlement – Validate` lists, per SKU, installed quantity, entitled quantity, and the position (over / under / balanced).

Test 5 — Exceptions are captured, not dropped

Do: Inspect the `Exception Register – Unmatched & Edge Cases – Validate` from `SWDISC-Report.ps1`. **Expected:** Every install with no SKU, and every SKU with installs but no entitlement, appears in the register with a reason — nothing silently discarded.

Test 6 — Remediation preview is safe

Do: Run `SWDISC-Remediation.ps1` (default `-WhatIf`). **Expected:** Pending normalization-map fixes are previewed only; no changes are committed until the script is re-run with `-WhatIf:$false`.

Test 7 — Position holds across sources

Do: After expansion (Step 10), re-run `SWDISC-Report.ps1` for the full scope. **Expected:** The ledger reconciles consistently across sources for shared SKUs; the same product from two sources counts once per device, not twice.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-6 pass for the pilot source; the exception register is triaged
Expand pass	Tests 1-5 and 7 pass for the full multi-source scope

6. Troubleshooting Guide — Software Discovery & Entitlement Reconciliation

Use `SWDISC-Discovery.ps1` and `SWDISC-Validation.ps1` to confirm inputs, `SWDISC-Report.ps1` for the position, and `SWDISC-Remediation.ps1` (default `-WhatIf`) for safe normalization-map fixes.

A. Discovery returns no installed software

Symptom	Cause	Remediation
Inventory file is empty	Software inventory not enabled on the source	Configuration Manager: enable Software/Hardware inventory + Asset Intelligence on the scoped collection. Intune: confirm `Apps > Monitor > Discovered apps` is populated
Intune path returns nothing	Missing Graph scope	Connect with `DeviceManagementManagedDevices.Read.All`; re-run `SWDISC-Discovery.ps1`
Configuration Manager path errors	SMS Provider/WMI not reachable	Confirm the console/cmdlets are installed and the `root\sms\site_<code>` namespace is reachable with a read role
Flexera rows missing	Export not placed where the script expects	Produce the installed-software export from the Flexera console and place it at the expected path; the script consumes the export, it does not call Flexera

B. Rows do not normalize to a SKU

Symptom	Cause	Remediation
Many unresolved rows in validation	Normalization map missing rules	Add rules to the `Normalization Map – SKU Vocabulary – Configure`; queue fixes and apply with `SWDISC-Remediation.ps1`
Same product appears as several SKUs	Edition/version suffixes not collapsed	Add rules that map edition/patch variants to one canonical SKU

Symptom	Cause	Remediation
Title resolves to the wrong SKU	Over-broad alias rule	Tighten the matching rule (publisher + title + edition), re-run `SWDISC-Validation.ps1`

C. Reconciliation ledger looks wrong

Symptom	Cause	Remediation
SKUs show installs but zero entitled	Entitlement register uses a different SKU vocabulary	Align register SKUs to the normalization map vocabulary so they join
Counts doubled for one product	Same install reported by two sources	Confirm the merge deduplicates on device plus SKU
Over-deployment false positive	Licensing metric not recorded (e.g. per-core)	Record the metric in the entitlement register; reconcile against the metric, not a raw count

D. Exceptions never shrink

- Each cycle, triage the `Exception Register – Unmatched & Edge Cases – Validate`: add a normalization

rule, record a missing entitlement, or flag a true gap — then re-run `SWDISC-Report.ps1` so resolved items leave the register.

E. Remediation made no change

- `SWDISC-Remediation.ps1` defaults to `-WhatIf` (preview only). Re-run with `-WhatIf:$false` to

commit, then confirm with `SWDISC-Validation.ps1`.

F. When to escalate

If discovery returns data, all rows normalize, the entitlement register shares the SKU vocabulary, and the ledger still disagrees with a known true-up, capture the inventory file, the normalization map, and the entitlement register, and review with procurement against the contract terms (downgrade rights, metric, edition coverage) before declaring a compliance gap.

7. Best Practices — Software Discovery & Entitlement Reconciliation

- **Pilot on one source first.** Prove the pipeline on a single discovery source or product family

before merging all sources — it surfaces normalization gaps cheaply and is the single most important practice.

• **One SKU vocabulary, everywhere.** The `Normalization Map – SKU Vocabulary – Configure` and the `Entitlement Register – Systems of Record – Configure` must use the same canonical SKUs, or the reconciliation join silently drops rows.

• **Never drop an unmatched install.** Every install that resolves to no SKU goes into the `Exception Register – Unmatched & Edge Cases – Validate` for a decision — exceptions are signal, not noise.

• **Treat the normalization map as a living asset.** Add a rule for each new product family as it appears; a stale map is the top cause of a noisy, untrustworthy ledger.

• **Deduplicate across sources.** When two sources report the same product on the same device, count it once — confirm the merge keys on device plus SKU.

• **Record the licensing metric.** Per-core, per-processor, and per-user-with-downgrade SKUs only reconcile correctly when the entitlement register stores the metric, not just a count.

• **Keep discovery read-only.** Discovery, validation, and reporting never change source systems; only `SWDISC-Remediation.ps1` writes, and it defaults to `-WhatIf`.

• **Reconcile on a cadence.** Run the pipeline each cycle and after every renewal or true-up so the position never drifts from reality.

• **Defer Flexera internals to the vendor.** Consume the Flexera installed-software export; do not reverse-engineer its recognition or API — its setup is the vendor's documentation, not this method.

• **Document the artifacts.** Record where each named artifact lives and who owns each exception decision, so the pipeline is repeatable and audit-ready.

8. Deliverables Summary — Software Discovery & Entitlement Reconciliation

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What discovery/reconciliation is, the named artifacts, the pipeline, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Asset Mgt)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: discover, normalize, import entitlements, reconcile, route exceptions, expand
5. Testing Guide	Validation procedures with pass/fail gates

Document	Purpose
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`SWDISC-Discovery.ps1`	Reads installed software from Intune detected apps (Graph) and Configuration Manager (Get-CM / WMI), or ingests a Flexera export, into the discovery inventory. Read-only
`SWDISC-Validation.ps1`	Confirms every raw row resolves through the normalization map to a SKU and that the entitlement register shares the SKU vocabulary. Read-only
`SWDISC-Report.ps1`	Builds the reconciliation ledger (installed vs entitled per SKU) and the exception register. Read-only
`SWDISC-Remediation.ps1`	Applies queued normalization-map fixes for unmatched titles; defaults to -WhatIf
`SWDISC-Set-ScopeTargeting.ps1`	Creates/confirms the pilot discovery-scope group (one source / one product family)

The named artifacts

Artifact	Stage
`Installed-Software Inventory – All Sources – Discovery`	Discovery
`Normalization Map – SKU Vocabulary – Configure`	Configure
`Entitlement Register – Systems of Record – Configure`	Configure
`Reconciliation Ledger – SKU vs Entitlement – Validate`	Validate
`Exception Register – Unmatched & Edge Cases – Validate`	Validate
`Discovery Scope Group – Pilot Source – Pilot`	Pilot

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.

- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Raw software discovery turned into a clean, reconciled license position — normalized, mapped to entitlements, exceptions flagged — proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SWDISC-Discovery.ps1

Purpose: Reads installed-software data from one or more discovery sources and writes the merged "Installed-Software Inventory - All Sources - Discovery" CSV (publisher, title, version, device, source). Read-only against all sources. - Intune: detected apps via Microsoft Graph (managedDeviceCount expanded to devices). - Configuration Manager: software inventory via Get-CM cmdlets, falling back to the SMS Provider WMI namespace (SMS_G_System_INSTALLED_SOFTWARE). - Flexera: ingests an installed-software export produced from the Flexera console. This script consumes that export file; it does NOT call any Flexera API (defer Flexera setup and export generation to Flexera's own documentation). HOW TO RUN Intune : powershell -ExecutionPolicy Bypass -File .\SWDISC-Discovery.ps1 -Source Intune -OutCsv .\inventory.csv (requires Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All) ConfigMgr: powershell -ExecutionPolicy Bypass -File .\SWDISC-Discovery.ps1 -Source ConfigMgr -SiteCode P01 -OutCsv .\inventory.csv Flexera : powershell -ExecutionPolicy Bypass -File .\SWDISC-Discovery.ps1 -Source Flexera -FlexeraExportCsv .\flexera-export.csv -OutCsv .\inventory.csv EXPECTED RESULTS A merged installed-software CSV with columns: Source, Device, Publisher, Title, Version. Re-run per source and merge to build the full "All Sources" inventory. .NOTES Workplace Marketplace — Asset Mgt. Read-only discovery. #> [CmdletBinding()] param([Parameter(Mandatory)][ValidateSet('Intune','ConfigMgr','Flexera')][string]$Source, [string]$OutCsv = ".\Installed-Software-Inventory.csv", [string]$SiteCode, [string]$FlexeraExportCsv)

How to run: Intune : powershell -ExecutionPolicy Bypass -File .\SWDISC-Discovery.ps1 -Source Intune -OutCsv .\inventory.csv (requires Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All) ConfigMgr: powershell -ExecutionPolicy Bypass -File .\SWDISC-Discovery.ps1 -Source ConfigMgr -SiteCode P01 -OutCsv .\inventory.csv Flexera : powershell -ExecutionPolicy Bypass -File .\SWDISC-Discovery.ps1 -Source Flexera -FlexeraExportCsv .\flexera-export.csv -OutCsv .\inventory.csv EXPECTED RESULTS A merged installed-software CSV with columns: Source, Device, Publisher, Title, Version. Re-run per source and merge to build the full "All Sources" inventory. .NOTES Workplace Marketplace — Asset Mgt. Read-only discovery. #> [CmdletBinding()] param([Parameter(Mandatory)][ValidateSet('Intune','ConfigMgr','Flexera')][string]$Source, [string]$OutCsv = ".\Installed-Software-Inventory.csv", [string]$SiteCode, [string]$FlexeraExportCsv)

SWDISC-Remediation.ps1

Purpose: Closes mapping gaps without touching any source system. It reads a queue of pending alias rules (Publisher, Title, SKU) — typically the unmatched rows triaged from the "Exception Register - Unmatched & Edge Cases - Validate" — and appends the new rules to the "Normalization Map - SKU Vocabulary - Configure", skipping any rule that already exists. SAFE BY DEFAULT: runs with -WhatIf and only previews the changes. Re-run with -WhatIf:$false to commit. A timestamped backup of the map is written before any write. HOW TO RUN Preview : powershell -ExecutionPolicy Bypass -File .\SWDISC-Remediation.ps1 ` -NormalizationMapCsv .\Normalization-Map.csv -PendingRulesCsv .\Pending-Rules.csv Commit : powershell -ExecutionPolicy Bypass -File .\SWDISC-Remediation.ps1 ` -NormalizationMapCsv .\Normalization-Map.csv -PendingRulesCsv .\Pending-Rules.csv -WhatIf:$false

Pending rules columns expected: Publisher, Title, SKU. EXPECTED RESULTS Preview: lists rules that WOULD be added. Commit: appends new rules and reports the count. Re-run SWDISC-Validation.ps1 afterward to confirm the previously unmatched rows now resolve. .NOTES Workplace Marketplace — Asset Mgt. Only this script writes; it defaults to -WhatIf. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param([Parameter(Mandatory)][string]$NormalizationMapCsv, [Parameter(Mandatory)][string]$PendingRulesCsv) # Default to preview unless the caller explicitly passes -WhatIf:$false. if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true } foreach ($f in @($NormalizationMapCsv,$PendingRulesCsv)) { if (-not (Test-Path $f)) { throw "Not found: $f" } }

How to run: Preview : powershell -ExecutionPolicy Bypass -File .\SWDISC-Remediation.ps1 `-NormalizationMapCsv .\Normalization-Map.csv -PendingRulesCsv .\Pending-Rules.csv Commit : powershell -ExecutionPolicy Bypass -File .\SWDISC-Remediation.ps1` -NormalizationMapCsv .\Normalization-Map.csv -PendingRulesCsv .\Pending-Rules.csv -WhatIf:$false Pending rules columns expected: Publisher, Title, SKU. EXPECTED RESULTS Preview: lists rules that WOULD be added. Commit: appends new rules and reports the count. Re-run SWDISC-Validation.ps1 afterward to confirm the previously unmatched rows now resolve. .NOTES Workplace Marketplace — Asset Mgt. Only this script writes; it defaults to -WhatIf. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param([Parameter(Mandatory)][string]$NormalizationMapCsv, [Parameter(Mandatory)][string]$PendingRulesCsv) # Default to preview unless the caller explicitly passes -WhatIf:$false. if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true } foreach ($f in @($NormalizationMapCsv,$PendingRulesCsv)) { if (-not (Test-Path $f)) { throw "Not found: $f" } }

SWDISC-Report.ps1

Purpose: Normalizes the discovery inventory to SKUs, deduplicates per device + SKU, aggregates the installed quantity per SKU, and matches it against the entitlement register to produce: - "Reconciliation Ledger - SKU vs Entitlement - Validate" (installed, entitled, position). - "Exception Register - Unmatched & Edge Cases - Validate" (installs with no SKU, and SKUs with installs but no entitlement). Read-only: reads CSVs and writes report CSVs; changes no source system. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SWDISC-Report.ps1 ` -InventoryCsv .\Installed-Software-Inventory.csv ` -NormalizationMapCsv .\Normalization-Map.csv ` -EntitlementRegisterCsv .\Entitlement-Register.csv ` -LedgerCsv .\Reconciliation-Ledger.csv ` -ExceptionCsv .\Exception-Register.csv Map columns: Publisher, Title, SKU. Register columns: SKU, EntitledQty, Metric. EXPECTED RESULTS A console summary of over/under/balanced SKUs, plus the ledger and exception-register CSVs. .NOTES Workplace Marketplace — Asset Mgt. Read-only reporting. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [Parameter(Mandatory)][string]$NormalizationMapCsv, [Parameter(Mandatory)][string]$EntitlementRegisterCsv, [string]$LedgerCsv = ".\Reconciliation-Ledger.csv", [string]$ExceptionCsv = ".\Exception-Register.csv") foreach ($f in @($InventoryCsv,$NormalizationMapCsv,$EntitlementRegisterCsv)) { if (-not (Test-Path $f)) { throw "Not found: $f" } }

How to run: powershell -ExecutionPolicy Bypass -File .\SWDISC-Report.ps1 `-InventoryCsv .\Installed-Software-Inventory.csv -NormalizationMapCsv .\Normalization-Map.csv -EntitlementRegisterCsv .\Entitlement-Register.csv` -LedgerCsv .\Reconciliation-Ledger.csv ` -ExceptionCsv .\Exception-Register.csv Map columns: Publisher, Title, SKU. Register columns: SKU, EntitledQty, Metric. EXPECTED RESULTS A console summary of over/under/balanced SKUs, plus the ledger and exception-register CSVs. .NOTES Workplace Marketplace — Asset Mgt. Read-only reporting. #> [CmdletBinding()] param(

[Parameter(Mandatory)][string]$InventoryCsv, [Parameter(Mandatory)][string]$NormalizationMapCsv, [Parameter(Mandatory)][string]$EntitlementRegisterCsv, [string]$LedgerCsv = ".\Reconciliation-Ledger.csv", [string]$ExceptionCsv = ".\Exception-Register.csv") foreach ($f in @($InventoryCsv,$NormalizationMapCsv,$EntitlementRegisterCsv)) { if (-not (Test-Path $f)) { throw "Not found: $f" } }

SWDISC-Set-ScopeTargeting.ps1

Purpose: Bounds the reconciliation pilot to one discovery source / one product family by ensuring an Entra ID security group exists to target the pilot devices, and records the chosen source and (optional) product family for the pipeline. Other scripts read this scope when running the pilot before fleet-wide reconciliation. Creating the group is the only write; pass -WhatIf to preview. Membership is left to your standard group-assignment process (manual or dynamic membership rule) so this script does not guess device membership. HOW TO RUN Preview : powershell -ExecutionPolicy Bypass -File .\SWDISC-Set-ScopeTargeting.ps1 ` -PilotSource Intune -ProductFamily "Adobe Acrobat" -WhatIf Create : powershell -ExecutionPolicy Bypass -File .\SWDISC-Set-ScopeTargeting.ps1 ` -PilotSource Intune -ProductFamily "Adobe Acrobat" Requires Microsoft Graph SDK and Group.ReadWrite.All to create the group. EXPECTED RESULTS Confirms (or creates) the security group "Discovery Scope Group - Pilot Source - Pilot" and prints the group id plus the recorded pilot source / product family. Add members via your normal process, then run SWDISC-Discovery.ps1 against the pilot source. .NOTES Workplace Marketplace — Asset Mgt. The only write is creating the Entra group (honors -WhatIf). #> [CmdletBinding(SupportsShouldProcess=$true)] param([Parameter(Mandatory)][ValidateSet('Intune','ConfigMgr','Flexera')][string]$PilotSource, [string]$ProductFamily, [string]$GroupName = "Discovery Scope Group - Pilot Source - Pilot") if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All' -NoWelcome

How to run: Preview : powershell -ExecutionPolicy Bypass -File .\SWDISC-Set-ScopeTargeting.ps1 `-PilotSource Intune -ProductFamily "Adobe Acrobat" -WhatIf Create : powershell -ExecutionPolicy Bypass -File .\SWDISC-Set-ScopeTargeting.ps1` -PilotSource Intune -ProductFamily "Adobe Acrobat" Requires Microsoft Graph SDK and Group.ReadWrite.All to create the group. EXPECTED RESULTS Confirms (or creates) the security group "Discovery Scope Group - Pilot Source - Pilot" and prints the group id plus the recorded pilot source / product family. Add members via your normal process, then run SWDISC-Discovery.ps1 against the pilot source. .NOTES Workplace Marketplace — Asset Mgt. The only write is creating the Entra group (honors -WhatIf). #> [CmdletBinding(SupportsShouldProcess=$true)] param([Parameter(Mandatory)][ValidateSet('Intune','ConfigMgr','Flexera')][string]$PilotSource, [string]$ProductFamily, [string]$GroupName = "Discovery Scope Group - Pilot Source - Pilot") if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All' -NoWelcome

SWDISC-Validation.ps1

Purpose: Validates the inputs to reconciliation: 1) Every row in the "Installed-Software Inventory - All Sources - Discovery" resolves through the "Normalization Map - SKU Vocabulary - Configure" to a canonical SKU. 2) The SKUs in the "Entitlement Register - Systems of Record - Configure" use the same SKU vocabulary as the normalization map (so the reconciliation join works). Read-only: reads CSVs and

reports; changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SWDISC-Validation.ps1 ` -InventoryCsv .\Installed-Software-Inventory.csv ` -NormalizationMapCsv .\Normalization-Map.csv ` -EntitlementRegisterCsv .\Entitlement-Register.csv Normalization map columns expected: Publisher, Title, Version (match key) and SKU (target). Entitlement register columns expected: SKU, EntitledQty, Metric. EXPECTED RESULTS "VALIDATION PASSED" when zero unresolved rows and zero orphaned register SKUs. Otherwise the unresolved rows and orphaned SKUs are listed for rule/vocabulary fixes. .NOTES Workplace Marketplace — Asset Mgt. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [Parameter(Mandatory)][string]$NormalizationMapCsv, [string]$EntitlementRegisterCsv) foreach ($f in @($InventoryCsv,$NormalizationMapCsv)) { if (-not (Test-Path $f)) { throw "Not found: $f" } }

How to run: powershell -ExecutionPolicy Bypass -File .\SWDISC-Validation.ps1 `-InventoryCsv .\Installed-Software-Inventory.csv` -NormalizationMapCsv .\Normalization-Map.csv ` -EntitlementRegisterCsv .\Entitlement-Register.csv Normalization map columns expected: Publisher, Title, Version (match key) and SKU (target). Entitlement register columns expected: SKU, EntitledQty, Metric. EXPECTED RESULTS "VALIDATION PASSED" when zero unresolved rows and zero orphaned register SKUs. Otherwise the unresolved rows and orphaned SKUs are listed for rule/vocabulary fixes. .NOTES Workplace Marketplace — Asset Mgt. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [Parameter(Mandatory)][string]$NormalizationMapCsv, [string]$EntitlementRegisterCsv) foreach ($f in @($InventoryCsv,$NormalizationMapCsv)) { if (-not (Test-Path $f)) { throw "Not found: $f" } }